Binding Boolean Expressions and Extended Pattern Matching

Arthur Charguéraud, Yanni Lefki

Inria Camus & University of Strasbourg, CNRS, ICube









Journée LVP, November 13, 2025

Motivation

Story of pattern matching:

- introduced with ML in the 1970's
- exhaustiveness checking in the 1980's
- efficient compilation scheme in the late 1990's
- first-class patterns in Racket/Scheme the 1990's
- Haskell views and F# active patterns in the 2000's
- expressive patterns also in Successor-ML in the mid-2000's

...still ongoing!

- Rust's construct: `if let PAT = EXPR { BODY }`, in RFC from 2014
- *Ultimate Conditional Syntax* by Cheng and Parreaux, OOPSLA'24

This work provides a yet slightly simpler presentation of a programming language that includes all the desirable features.

Feature focus #1: smart deconstructors

Smart constructors

```
type trm = {
  desc : desc;
  loc : location option; (* from parser *)
  typ : typ option; } (* from typer *)

and desc =
  | Trm_var of var
  | Trm_bool of bool
  | Trm_if of trm * trm * trm
  | Trm_and of trm * trm
  ...


(* To construct a term in a code transformation, we would write e.g.:
  { desc = Trm_if (t1,t2,t3); loc = None; typ = None }.
  Instead, one can use smart constructors such as [trm_if] and [trm_and]. *)

let trm_make (d : desc): trm =
  { desc = d; loc = None; typ = None }

let trm_if (t1 : trm) (t2 : trm) (t3 : trm) : trm =
  trm_make (Trm_if (t1,t2,t3))
let trm_and (t1 : trm) (t2 : trm) : trm =
  trm_make (Trm_and (t1,t2))
```

Benefits of smart constructors

Smart-constructors offer a clean API for building AST terms, based solely on functions, not exposing the constructors.

Say we decide to remove `Trm_and` and encode this construct using an `if`:

```
 and desc =
   | Trm_var of var
   | Trm_bool of bool
   | Trm_if of trm * trm * trm
(* | Trm_and of trm * trm <-- removed *)

 (* updated version of the smart constructor for [trm_and] *)
 let trm_and (t1 : trm) (t2 : trm) : trm =
   trm_if (t1, t2, trm_bool false)
```

No need to modify the client code, which continues to call `trm_and`.

What about AST deconstruction?

Client function to recognize terms of the form `t1 && t2 && t3`.

```
let trm_and_3_inv (t : trm) : (trm * trm * trm) option =
  begin match t.desc with
  | Trm_and (t1, { desc = Trm_and (t2,t3); _ }) -> Some (t1,t2,t3)
  | _ -> None
  end
```

This client code mentions the constructor `Trm_and`.

Such an explicit manipulation of constructors suffers from:

1. **Lack of abstraction**
   → e.g., if we remove `Trm_and`, all the client code needs to change
2. **Lack of conciseness**
   → compared with the construction: `trm_and t1 (trm_and t2 t3)`

Smart deconstructors = views = active patterns

```
(* [REVISITED] Function to recognize terms of the form [t1 && t2 && t3].
  Inside patterns, [trm_and] is short for [Pattern.trm_and] *)
let trm_and_3_inv (t : trm) : (trm * trm * trm) option =
  match t with
  | trm_and t1 (trm_and t2 t3) -> Some (t1,t2,t3)
  | _ -> None

(* Definition of the smart deconstructors *)
let Pattern.trm_if (t : trm) : (trm * trm * trm) option =
  match t.desc with
  | Trm_if t1 t2 t3 -> Some (t1,t2,t3)
  | _ -> None

let Pattern.trm_bool (t : trm) : bool option =
  match t.desc with
  | Trm_bool b -> Some b
  | _ -> None

(* [Pattern.trm_and], assuming [trm_and] is encoded using a conditional. *)
let Pattern.trm_and (t : trm) : (trm * trm) option =
  match t with
  | trm_if t1 t2 (trm_bool false) -> Some (t1,t2)
  | _ -> None
```

Feature focus #2: tests that export bindings

Limitation of when-clauses

Idiomatic code, yet with inefficient map operations:

```
match r with
| Some k when Hashtbl.mem tbl k ->
    let v = Hashtbl.get tbl k in
    f v
| None ->
    a_big_expression
```

Efficient code, yet more verbose and with unpleasant flow:

```
let cont () =
  a_big_expression in
match r with
| Some k ->
    begin match Hashtbl.get_opt tbl k with
    | Some v -> f v
    | None -> cont()
    end
| _ -> cont()
```

Binding boolean expressions (BBEs)

Efficient and concise:

```
match r with
| Some k when Hashtbl.get_opt tbl k is Some v -> f v
| _ -> a_big_expression
```

The construct "`t is p`" is a *binding-boolean-expression*: it computes a boolean value *and* binds pattern variables in case the pattern matches.

Same as above, using a BBEs in a conditional:

```
if (r is Some k) && (Hashtbl.get_opt tbl k is Some v)
  then f v
  else a_big_expression
```

Related work: example taken from Rust RFC 2497-2025-06-18.

```
if let Some((fn_name, aft_name)) = s.split_once("(")
  && !fn_name.is_empty()
  && is_legal_ident(fn_name)
  && let Some((args_str, "")) = aft_name.rsplit_once(")") {
   return fn_name + args_str;
}
```

Formalization

Syntax of terms

A λ-calculus with binding-boolean-expressions, written b.

t, f, g	:=	x	variable
	$\mid$	$\lambda(x_1, ..., x_n).t$	λ-abstraction
	$\mid$	$f\ (t_1, ..., t_n)$	application
	$\mid$	$\mathbf{let}\, x\ =\ t_1\, \mathbf{in}\, t_2$	let-binding
	$\mid$	$\mathbf{if}\, b\, \mathbf{then}\, t_1\, \mathbf{else}\, t_2$	conditional

The match construct

Encoding for **match**:

$$
\begin{array}{l}
\textbf{match}\, t\, \textbf{with} \\
\quad \mid p_1 \rightarrow\ t_1 \\
\quad \mid p_2 \rightarrow\ t_2
\end{array}
\quad \Longrightarrow \quad
\begin{array}{l}
\textbf{if}\, (x\, \textbf{is}\, p_1)\, \textbf{then}\, t_1\, \textbf{else} \\
\textbf{if}\, (x\, \textbf{is}\, p_2)\, \textbf{then}\, t_2\, \textbf{else} \\
\textbf{raise}\, \mathsf{Match_failure}
\end{array}
$$

Semantics:

- Like in OCaml, clauses must be evaluated in order.
- Unlike in OCaml, when-clauses can appear in depth, and arbitrary effects are allowed in when-clauses and smart-deconstructors.

Syntax of binding-boolean-expressions

b :=	|	t **is** p	pattern-matching	binds $\mathcal{B}(p)$
	|	b_1 && b_2	conjunction with b_1 binding into b_2	binds $\mathcal{B}(b_1) \cup \mathcal{B}(b_2)$
	|	b_1 || b_2	disjunction	binds $\mathcal{B}(b_1) \cap \mathcal{B}(b_2)$
	|	**not** b	negation	binds $\varnothing$

where p is a *pattern*, and where $\mathcal{B}$ means "bindings exported by".

Note: the `&&`, `||` and **not** constructs can be encoded using a construct named $\mathbf{if}^{\mathrm{bbe}}$.

Syntax of patterns

p	:=	$x^?$	pattern variable	binds $\{x\}$
	$\mid$	$p_1 \mid p_2$	pattern disjunction	binds $\mathcal{B}(p_1) \cap \mathcal{B}(p_2)$
	$\mid$	$p_1 \,\&\, p_2$	pattern intersection	binds $\mathcal{B}(p_1) \cup \mathcal{B}(p_2)$
	$\mid$	p **when** b	guarded pattern with p binding into b	binds $\mathcal{B}(p) \cup \mathcal{B}(b)$
	$\mid$	$f\ (p_1, ..., p_n)$	inversor pattern	binds $\bigcup_i \mathcal{B}(p_i)$
	$\mid$	g	predicate pattern	binds $\varnothing$

where :

- f is a smart deconstructor of type: $T \to (T_1, ..., T_n)$ option.
  $f\ (p_1, ..., p_n)$ is equivalent to: $x^?$ **when** $f\,(x)$ **is** Some $(p_1, ..., p_n)$.
- classic data constructors can be viewed as smart deconstructors
- g is a boolean function of type: $T \to$ bool.
  g is equivalent to: $x^?$ **when** $g\,(x)$.

Note: the `&` and `|` pattern constructs can both be encoded using **when** and BBE conditionals.

Typing & Semantics

Typing judgments, Typing rules for terms and BBEs

- $E \vdash_{\mathsf{trm}} t : T$
- $E \vdash_{\mathsf{bbe}} b \rightsquigarrow B$
- $E \vdash_{\mathsf{pat}} p : T \rightsquigarrow B$

- E maps context variables to types
- B maps exported variables to types
- Not checking for exhaustiveness

$$\frac{E \vdash_{\mathsf{bbe}} b \rightsquigarrow B \qquad E, B \vdash_{\mathsf{trm}} t_1 : T \qquad E \vdash_{\mathsf{trm}} t_2 : T}{E \vdash_{\mathsf{trm}} \textbf{if } b \textbf{ then } t_1 \textbf{ else } t_2 : T}$$

$$\frac{E \vdash_{\mathsf{trm}} t : T \qquad E \vdash_{\mathsf{pat}} p : T \rightsquigarrow B}{E \vdash_{\mathsf{bbe}} t \textbf{ is } p \rightsquigarrow B}$$

$$\frac{E \vdash_{\mathsf{bbe}} b_1 \rightsquigarrow B_1 \qquad E, B_1 \vdash_{\mathsf{bbe}} b_2 \rightsquigarrow B_2 \qquad B_1 \# B_2}{E \vdash_{\mathsf{bbe}} b_1 \,\&\&\, b_2 \rightsquigarrow B_1 \uplus B_2}$$

where $B_1 \# B_2$ asserts disjointness. We disallow shadowing in `&&`.

Typing rules for patterns

Recall that $E \vdash_{\mathsf{pat}} p : T \rightsquigarrow B$ means that p exports the bindings B.

$$\frac{}{E \vdash_{\mathsf{pat}} x^{?} \ : \ T \ \rightsquigarrow \ \{x : T\}}$$

$$\frac{E \vdash_{\mathsf{pat}} p_1 : T \rightsquigarrow B_1 \qquad E \vdash_{\mathsf{pat}} p_2 : T \rightsquigarrow B_2 \qquad B_1 \# B_2}{E \vdash_{\mathsf{pat}} (p_1 \,\&\, p_2) \ : \ T \ \rightsquigarrow \ B_1 \uplus B_2}$$

$$\frac{E \vdash_{\mathsf{pat}} p_1 : T \rightsquigarrow B \qquad E \vdash_{\mathsf{pat}} p_2 : T \rightsquigarrow B}{E \vdash_{\mathsf{pat}} (p_1 \mid p_2) \ : \ T \ \rightsquigarrow \ B}$$

$$\frac{\begin{array}{c} E \vdash_{\mathsf{trm}} f : T \to (T_1, ..., T_n)\ \mathsf{option} \\ \forall i. \quad E \vdash_{\mathsf{pat}} p_i : T_i \rightsquigarrow B_i \qquad \forall i \neq j.\ B_i \# B_j \end{array}}{E \vdash_{\mathsf{pat}} f\ (p_1, ..., p_n) \ : \ T \ \rightsquigarrow \ \bigcup_i B_i}$$

$$\frac{E \vdash_{\mathsf{trm}} g : T \to \mathsf{bool}}{E \vdash_{\mathsf{pat}} g \ : \ T \ \rightsquigarrow \ \varnothing}$$

$$\frac{E \vdash_{\mathsf{pat}} p : T \rightsquigarrow B_1 \qquad E, B_1 \vdash_{\mathsf{bbe}} b \rightsquigarrow B_2 \qquad B_1 \# B_2}{E \vdash_{\mathsf{pat}} (p \ \textbf{when} \ b) \ : \ T \ \rightsquigarrow \ B_1 \uplus B_2}$$

Evaluation judgments, rules for terms and BBEs

- $t \Downarrow_{\mathsf{trm}} v$
- $b \Downarrow_{\mathsf{bbe}} r$
- $v \triangleright p \Downarrow_{\mathsf{pat}} r$

- r := Mismatch | Match M
- M maps variables to values

Mutable store not shown

$$\frac{b \Downarrow_{\mathsf{bbe}} \mathsf{Mismatch} \qquad t_2 \Downarrow_{\mathsf{trm}} v}{\textbf{if } b \textbf{ then } t_1 \textbf{ else } t_2 \ \Downarrow_{\mathsf{trm}} \ v}$$

$$\frac{b \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ M \qquad \mathsf{Subst}(M, t_1) \Downarrow_{\mathsf{trm}} v}{\textbf{if } b \textbf{ then } t_1 \textbf{ else } t_2 \ \Downarrow_{\mathsf{trm}} \ v}$$

$$\frac{t \Downarrow v \qquad v \triangleright p \Downarrow_{\mathsf{pat}} r}{t \textbf{ is } p \ \Downarrow_{\mathsf{bbe}} \ r}$$

$$\frac{b_1 \Downarrow_{\mathsf{bbe}} \mathsf{Mismatch}}{b_1 \,\&\&\, b_2 \Downarrow_{\mathsf{bbe}} \mathsf{Mismatch}}$$

$$\frac{b_1 \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ M_1 \qquad \mathsf{Subst}(M_1, b_2) \Downarrow_{\mathsf{bbe}} \mathsf{Mismatch}}{b_1 \,\&\&\, b_2 \Downarrow_{\mathsf{bbe}} \mathsf{Mismatch}}$$

$$\frac{b_1 \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ M_1 \qquad \mathsf{Subst}(M_1, b_2) \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ M_2 \qquad M_1 \# M_2}{b_1 \,\&\&\, b_2 \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ (M_1 \uplus M_2)}$$

$$\frac{b_1 \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ M_1}{b_1 \,||\, b_2 \Downarrow_{\mathsf{bbe}} \mathsf{Match}\ M_1}$$

$$\frac{b_1 \Downarrow_{\mathsf{bbe}} \mathsf{Mismatch} \qquad b_2 \Downarrow_{\mathsf{bbe}} r}{b_1 \,||\, b_2 \Downarrow_{\mathsf{bbe}} r}$$

Evaluation rules for patterns

$v \triangleright p \Downarrow_{\mathsf{pat}} r$ tests whether v matches p, returns Mismatch or Match M.

$$\frac{}{v \triangleright x^{?} \Downarrow_{\mathsf{pat}} \text{Match } \{x \mapsto v\}}$$

$$\frac{v \triangleright p \Downarrow_{\mathsf{pat}} \text{Mismatch}}{v \triangleright p \textbf{ when } b \Downarrow_{\mathsf{pat}} \text{Mismatch}}$$

$$\frac{v \triangleright p \Downarrow_{\mathsf{pat}} \text{Match } M \quad \text{Subst}(M, b) \Downarrow_{\mathsf{bbe}} \text{Mismatch}}{v \triangleright p \textbf{ when } b \Downarrow_{\mathsf{pat}} \text{Mismatch}}$$

$$\frac{v \triangleright p \Downarrow_{\mathsf{pat}} \text{Match } M_1 \quad \text{Subst}(M_1, b) \Downarrow_{\mathsf{bbe}} \text{Match } M_2 \qquad M_1 \# M_2}{v \triangleright p \textbf{ when } b \Downarrow_{\mathsf{pat}} \text{Match } (M_1 \uplus M_2)}$$

$$\frac{f(v) \Downarrow_{\mathsf{trm}} \text{None}}{v \triangleright f(p_1, ..., p_n) \Downarrow_{\mathsf{pat}} \text{Mismatch}}$$

$$\frac{f(v) \Downarrow_{\mathsf{trm}} \text{Some}(v_1, ..., v_n) \qquad \exists i.\ v_i \triangleright p_i \Downarrow_{\mathsf{pat}} \text{Mismatch}}{v \triangleright f(p_1, ..., p_n) \Downarrow_{\mathsf{pat}} \text{Mismatch}}$$

$$\frac{f(v) \Downarrow_{\mathsf{trm}} \text{Some}(v_1, ..., v_n) \qquad \forall i.\ \ v_i \triangleright p_i \Downarrow_{\mathsf{pat}} \text{Match } M_i \qquad \forall i \neq j.\ M_i \# M_j}{v \triangleright f(p_1, ..., p_n) \Downarrow_{\mathsf{pat}} \text{Match } (\bigcup_i M_i)}$$

$$\frac{g(v) \Downarrow_{\mathsf{trm}} \text{false}}{v \triangleright g \Downarrow_{\mathsf{pat}} \text{Mismatch}}$$

$$\frac{g(v) \Downarrow_{\mathsf{trm}} \text{true}}{v \triangleright g \Downarrow_{\mathsf{pat}} \text{Match } \varnothing}$$

Type soundness theorem

Theorem (Preservation)

1. $(t \Downarrow_{trm} v) \ \wedge \ (\vdash_{trm} t : T) \ \Rightarrow \ (\vdash_{trm} v : T)$
2. $(b \Downarrow_{bbe} \mathit{Match}\ M) \ \wedge \ (\vdash_{bbe} b \rightsquigarrow B) \ \Rightarrow \ (\vdash_{map} M : B)$
3. $(v \triangleright p \Downarrow_{pat} \mathit{Match}\ M) \ \wedge \ (\vdash_{pat} p : T \rightsquigarrow B) \ \wedge \ (\vdash_{trm} v : T)$
   $\Rightarrow \ (\vdash_{map} M : B)$

where $\vdash_{\mathsf{map}} M : B$ is defined as:

$$\mathsf{dom}\, M = \mathsf{dom}\, B \ \wedge \ \forall\, x \in \mathsf{dom}\, M.\ \vdash_{\mathsf{trm}} M(x) : B(x)$$

Future work: prove *progress*, unless branching into a non-existant else-branch.

Future work: Rocq formalization.

Conclusion & Future work

Conclusion & Future work

We are working towards an expressive, conservative extension of ML conditionals and pattern matching, with simple typing and semantics.

Formalization in Rocq would be nice.

Implementation:

- to be integrated into the OptiTrust framework for source-to-source transformation; specifying which rewrite rules preserve the semantics
- perhaps also as a standalone pre-processor for OCaml?
- or maybe even as a conservative language extension?

Question:

- Are all other practical programming patterns covered?